

Time Complexity

AND HOW TO APPROACH IT

CS1101S

BY EUGENE OH

Note: I am aware some of the mathematical portions are slightly off, I think a few off by 1 errors, but I lazy and it is sufficient to get the idea across

What is the problem size

- Problem size is some property of the input parameters, typically one that affects run time or space complexity (it is kind of vague but big O is meant to be an estimation/gauge of efficiency)
- Common problem sizes
 - Lists
 - Number of elements in the list / Number of pairs that make up the list
 - Trees (A few possible problem sizes)
 - Number of nodes in the tree (the length of the list if you flatten the tree into a list)
 - Depth of tree (how many nested lists in the tree)
 - Strings
 - Number of characters in a string
 - Numbers
 - Number of digits in a number
- If there are multiple input parameters, then problem size might be some formula involving properties of some/all input parameters
 - For $f(\text{list1}, \text{list2})$ could have a time complexity $O(\text{len1} * \text{len2})$ where len1 is length of list 1, len2 is length of list 2

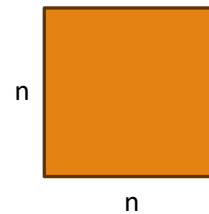
Basic $O(n)$

- Basic example would be a recursive function $f(n)$ that:
 1. Recursively calls $f(n-a)$ until some base case (where a is any constant)
 2. The runtime complexity of $f(n)$ excluding the recursive call is $O(1)$
- Many built-in functions for lists are $O(n)$
 - `append`, `enum_list`, `length`, `list_ref`
- Also take note that if $O(n)$ function calls are running one after another, the result would also be $O(n)$
 - Trick questions might include lines like `append(xs, list(length(xs)))`, with nested functions with $O(n)$ time complexity
 - Notice how you can rewrite this as:
 - `const len = length(xs)`
 - `append(xs, list(len))`
 - Which are both $O(n)$ and since they run one after another, it remains $O(n)$

$O(n^2)$, $O(n^3)$, ...

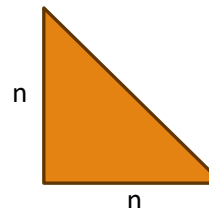
- Basic example for $O(n^2)$ running a $O(n)$ function n times, like a recursive function that:
 1. Recursively calls $f(n-a)$ until some base case (where a is any constant)
 2. The runtime complexity of $f(n)$ excluding the recursive call is $O(n)$
- Other cases involve passing accumulate, map and filter an input function with $O(n)$ runtime complexity
- One way to visualise this is through shapes, graphing the number of steps each iteration takes, time complexity would then be the area (for $O(n^2)$) or volume (for $O(n^3)$)
- Larger powers of form n^k simply have iterations that call lower power functions ($O(n^3)$ would iterate a $O(n^2)$ function, n times)

```
function f(xs) {  
  return map(x => length(xs), xs);  
}
```



Map runs n times
Each time, length runs
for n steps

```
function f(x) {  
  return x === 1  
    ? 1  
    : length(enum_list(1, x)) + f(x-1);  
}
```



Recursively calls itself n times
Each time, input parameter decreases, length runs for $n-k$ steps, enum_list also runs for $n-k$ steps where k is number of previous iterations
Input parameter decreases linearly from n to 1, visually forming a triangle

$O(\log n)$

- Basic example is a recursive function that:
 1. Recursively calls $f(n*a)$ until some base case (where a is any constant)
 2. The runtime complexity of $f(n)$ excluding the recursive call is $O(1)$
- This is due to the number of iterations not being linearly related to n , but instead logarithmically related
- The most common case is when input arguments are halved in size, however all scaling factors are possible and will lead to a $O(\log n)$ time complexity

```
function f(x) {  
    return x <= 1 ? 1 : 1 + f(x/2);  
}
```

Let S be the number of steps to run $f(n)$
Consider the base case and how initial n and S are related to final input argument.

$$1 \leq n * (0.5)^S$$
$$2^S \leq n$$
$$S \log 2 \leq \log n$$

From here we can see S is related to $\log n$

```
function f(x) {  
    return x >= 1000 ? 1 : 1 + f(x*10);  
}
```

Same as previous, Let S be the number of steps to run $f(n)$

$$1000 \leq n * (10)^S$$
$$\log(1000) \leq S + \log(n)$$
$$3 - \log(n) \leq S$$

From here we can see S is related to $\log n$, though in this case problem size would be better defined as $1/n$ (I think)

$O(n \log n)$

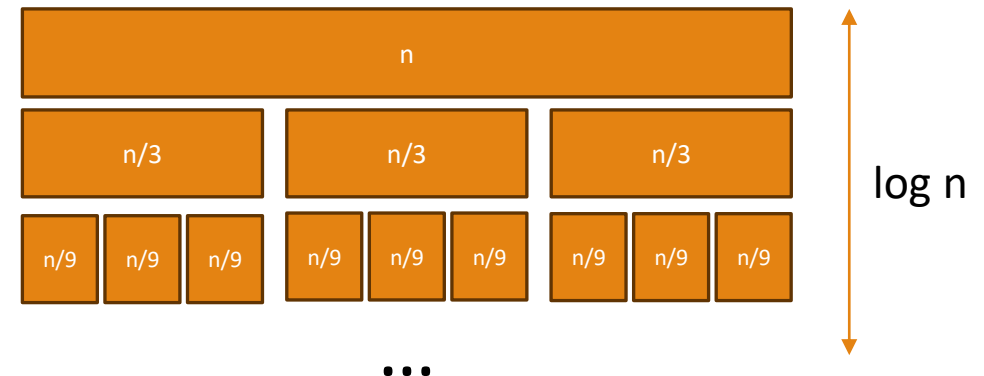
- Remember how for $O(n^2)$ is iterating an $O(n)$ function n times, similarly, iterating an $O(n)$ function $\log n$ times, leads to an $O(n \log n)$ function
- Merge sort is one example (though not exactly iterating an $O(n)$ function but something that is mathematically equivalent)
- Similar functions would also possess similar runtime complexities

```
function f(x) {  
  return x <= 1  
    ? 1  
    : length(enum_list(1,x)) + f(x/3) + f(x/3) + f(x/3);  
}
```

Notice that the steps taken by the sum of all 3 recursive calls is equal to the current function call

Each “Iteration” is thus $O(n)$, repeated for $\log n$ iterations. (Same derivation as $O(\log n)$)

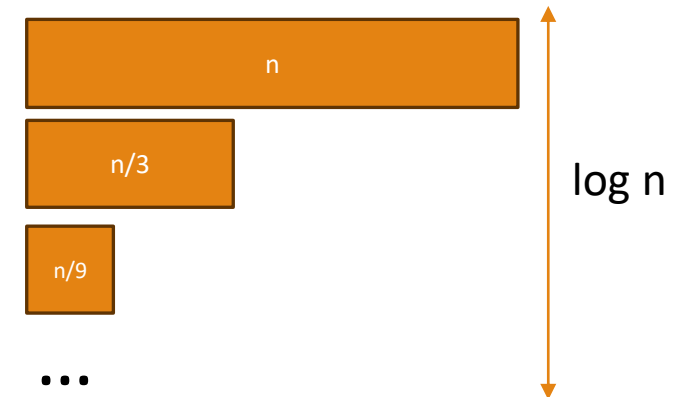
In terms of area visualisation, it would be $n \times \log n$ rectangle



$O(n)$, infinite series

- Consider if instead of repeated function calls for the earlier example, it was only recursively called once
- Some would assume $O(n \log n)$ since it is still iterating an $O(n)$ function $\log n$ times, however this fails to consider how each subsequent iteration has fewer steps, following a geometric progression
- Steps:
 - $n + n/3 + n/9 + \dots$ (For large n , we will just treat this as an infinite series)
 - $n * (1 + 1/3 + 1/9 + \dots)$ (Notice that this is a geometric series summation)
 - $n * (1 / (1 - 1/3))$ (Summation of infinite geometric series)
 - $3n/2$
- We can treat it as an infinite series as the trailing terms become negligible as n grows very large
- The relation between steps and n is linear so $O(n)$

```
function f(x) {  
  return x <= 1  
    ? 1  
    : length(enum_list(1,x)) + f(x/3);  
}
```



$O(n)$, geometric tree recursion

- Consider again the $O(n \log n)$ example, but now instead every iteration is $O(1)$
- Some would now assume $O(\log n)$, since it is $\log n$ iterations of a $O(1)$ function, however this fails to consider that the multiple function calls will lead to exponentially many steps each iteration
- Steps
 - $1, 3, 9, 27, \dots, 3^{\log_3 n - 1}$ (We cannot treat this as an infinite series, since the last term is always the most significant)
 - $1 * (3^{\log_3 n} - 1) / (3 - 1)$, (Summation of geometric series)
 - $(n - 1) / 2$, ($3^{\log_3 n} = n$)
- The relation between steps and n is linear so $O(n)$

```
function f(x) {  
  return x <= 1  
    ? 1  
    : f(x/3) + f(x/3) + f(x/3);  
}
```


$O(k^n)$, arithmetic tree recursion

- Now instead of scaling the problem size by a factor, it is instead subtraction
- Some would now assume $O(n)$, since it is n iterations of a $O(1)$ function, however similar to before this fails to consider that the multiple function calls will lead to exponentially many steps each iteration
- Steps
 - $1, 3, 9, 27, \dots, 3^{n-1}$ (We cannot treat this as an infinite series, since the last term is always the most significant)
 - $1 * (3^n - 1) / (3 - 1)$, (Summation of geometric series)
 - $(3^n - 1) / 2$,
- The relation between steps and n is exponential so $O(3^n)$ or more generally $O(k^n)$
- The difference with the previous example is the number of iterations

```
function f(x) {  
  return x <= 1  
    ? 1  
    : f(x-1) + f(x-1) + 1;  
}
```